



Least Authority
PRIVACY MATTERS

Midnight.js
Security Audit Report

Midnight Network

Final Audit Report: 11 March 2026

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[System Design](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Issue A: Private Data Is Stored Without Encryption \(Known Issue\)](#)

[Issue B: Unproven Transaction Is Passed to Wallet Provider](#)

[Suggestions](#)

[Suggestion 1: Explicitly Disable Debugging Code by Default](#)

[Suggestion 2: Document the Errors Thrown by the Midnight.js API and Providers](#)

[Suggestion 3: Document the Behavior of Transaction Submission Methods When the Transaction Fails](#)

[Suggestion 4: Document the Behavior of Transaction Submission Methods When the Transaction Does Not Appear](#)

[Suggestion 5: Zeroize Sensitive Data to Avoid Accidental Exposure](#)

[Suggestion 6: Use Monads to Distinguish Between Absent and Undefined Values](#)

[Suggestion 7: Minimize Exposure of Private Data to Application Code](#)

[Suggestion 8: Document the Framework's Threat Model](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

Shielded has requested Least Authority perform multiple audits of select components of the Midnight Network: the Ledger, the Node, the Compiler, Midnight.js, the Wallet API, and the Indexer. Midnight is a fourth-generation blockchain harnessing the power of zero-knowledge cryptography. This audit focuses on Midnight.js.

Project Dates

- **October 28, 2025 - November 6, 2025:** Initial Code Review (*Completed*)
- **November 10, 2025:** Delivery of Initial Audit Report (*Completed*)
- **March 11, 2026:** Verification Review (*Completed*)
- **March 11, 2026:** Delivery of Final Audit Report (*Completed*)

Review Team

- Paul Lorenc, Security Researcher and Engineer
- Eduardo Morais, Cryptography Researcher and Engineer
- Michael Rogers, Security Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of Midnight's Midnight.js followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repositories are considered in scope for the review:

- Midnight-js:
<https://github.com/midnightntwrk/midnight-js>

Specifically, we examined the Git revision for our initial review:

- 4255f7bd0b3eb21449598049456d00976f7bbe86

For the verification, we examined the Git revision:

- bdea44567255e618de1af912b1a1262c803ca4d1

For the review, this repository was cloned for use during the audit and for reference in this report:

- Midnight-js:
<https://github.com/LeastAuthority/midnight-js>

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- Websites:
 - <https://midnight.network>
 - <https://shielded.io>
- AuditList.ods (shared with Least Authority via email on 4 August 2025)

In addition, this audit report references the following documents:

- Transaction semantics:
<https://docs.midnight.network/develop/how-midnight-works/semantics>
- sanitize-data:
<https://github.com/jaktestowac/sanitize-data>
- wasm-zeroize:
<https://github.com/ForgeKey/wasm-zeroize>
- Zcash wallet app documentation:
https://zcash.readthedocs.io/en/latest/rtd_pages/wallet_threat_model.html

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Whether requests are passed correctly to the network core;
- Key management, including secure private key storage and management of encryption and signing keys;
- Denial of Service (DoS) and other security exploits that would impact the intended use or disrupt the execution;
- Protection against malicious attacks and other ways to exploit;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

Midnight.js is an application framework for developing applications that use the Midnight blockchain. It provides an API that allows applications to interface with Midnight-compatible wallets and with local or remote services that can create, store, and execute smart contracts; store and retrieve private data for contract use; compute zero-knowledge proofs; and communicate with blockchain nodes. The framework employs the provider design pattern, allowing applications to customize the implementation of these services if required.

Midnight.js is implemented in TypeScript. The default provider implementations use LevelDB for local storage of private data and HTTP(S) for communication with a proof server, which can be hosted either locally or remotely.

System Design

Our team reviewed Midnight.js and found that the Midnight development team has clearly prioritized security in both its design and implementation. The repository's overall structure is organized into a set of providers, which represent objects supplying data to library functions, and a `contracts` directory, which contains functions for interacting with Midnight transactions and submitting them to the blockchain. This separation of providers and contract functions is an effective approach to promoting clarity and maintainability in the library's architecture. We recommend that the Midnight team adopt the same design pattern when adding support for encrypted local storage ([Issue A](#)).

Our review focused on the `packages` directory of the repository, while the tests in the `testkit-js` directory were primarily referenced to understand the corresponding code. Within the `packages` directory, we reviewed the `compact` package, which provides utilities for installing and running the `compactc` smart contract compiler, and found no issues. The `types` and `utils` packages respectively contain type declarations used by Midnight.js and utility functions shared across components of the codebase. We found no issues in these packages, although [Suggestion 7](#) is relevant to the types in the `types` package.

The `fetch-zk-config-provider`, `http-client-proof-provider`, `indexer-public-data-provider`, and `node-zk-config-provider` packages contain relatively straightforward provider implementations, in which we did not identify any issues. The `level-private-state-provider` package contains another provider with a known issue, namely the lack of encryption for data at rest ([Issue A](#)). [Suggestion 5](#) also applies to this private state provider, while [Suggestion 2](#) and [Suggestion 8](#) are relevant to all providers as well as other parts of the codebase.

Most of the complexity in the codebase lies in the `contracts` package, which was the primary focus of our review. This package contains code for deploying, calling, and managing smart contracts. [Issue B](#) and the majority of the suggestions in this report relate to this package.

Each audited component includes a corresponding test suite located in a related test directory, which verifies the fundamental behavior of the component. While we found no issues with the component tests, our team noted that several tests in the `contracts` package call functions that return private data, and we recommend adopting the established provider pattern instead ([Suggestion 7](#)).

Dependencies

We examined the implemented dependencies in the codebase and found that both internal and external dependencies are managed with Yarn. Our review did not find any dependencies with known vulnerabilities.

Code Quality

We performed a manual review of the repositories in scope and found the code to be exemplary. In particular, it is well structured at the macro and micro levels, with rigorous use of types that promote type safety and improve readability.

Tests

The repositories in scope include extensive unit and integration tests that cover both error cases as well as the happy path. Additionally, test coverage is calculated and displayed automatically.

Documentation and Code Comments

The codebase is well commented with both inline and documentation comments. API documentation is automatically generated from these comments and made available both as Markdown files in the repository and on the project's website. Areas of the code requiring further attention or improvement are marked with TODO comments.

However, we found that errors thrown by certain methods and functions are not consistently documented ([Suggestion 2](#)).

Scope

The initial scope of the audit encompassed the entire `midnight-js` repository. However, due to the project's rapid development following the initial estimate, our team and the Midnight team agreed to exclude the `compact-js` and `platform-js` components from the scope. These components are intended for application use but are not yet integrated with other parts of `midnight-js`, allowing their exclusion without affecting the remainder of the audit.

Specific Issues & Suggestions

We list the issues and suggestions found during the review, in the order we reported them. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Issue A: Private Data Is Stored Without Encryption (Known Issue)	Medium	Partially Resolved
Issue B: Unproven Transaction Is Passed to Wallet Provider	Medium	Resolved
Suggestion 1: Explicitly Disable Debugging Code by Default	Informational	Resolved
Suggestion 2: Document the Errors Thrown by the Midnight.js API and Providers	Informational	Resolved
Suggestion 3: Document the Behavior of Transaction Submission Methods When the Transaction Fails	Informational	Resolved
Suggestion 4: Document the Behavior of Transaction Submission Methods When the Transaction Does Not Appear	Informational	Resolved
Suggestion 5: Zeroize Sensitive Data to Avoid Accidental Exposure	Informational	Unresolved
Suggestion 6: Use Monads to Distinguish Between Absent and Undefined Values	Informational	Unresolved
Suggestion 7: Minimize Exposure of Private Data to Application Code	Informational	Partially Resolved
Suggestion 8: Document the Framework's Threat Model	Informational	Unresolved

Issue A: Private Data Is Stored Without Encryption (Known Issue)

Location

[README.md?plain=1#L138](#)

[packages/level-private-state-provider/src/level-private-state-provider.ts](#)

Synopsis

The default implementation of the private state provider, which stores and retrieves private data for use in contracts, does not encrypt data at rest.

Impact

High.

The data stored by the private state provider includes signing keys for contracts and any data used in the private state of contracts. An adversary with access to this data could assume control of smart contracts with the same level of access as their creators and could observe information that Midnight's zero-knowledge proofs were designed to keep private.

Feasibility

Low.

The data managed by the private state provider is stored in the device's filesystem. On desktop and server platforms, this allows access to any process running under the same user's identity or one with elevated privileges (root/administrator). On mobile platforms, access is more restricted due to application sandboxing, reducing the overall feasibility of an attack.

Severity

Medium.

Technical Details

The absence of encryption for data at rest is a known issue that was identified by the Midnight team prior to the start of the audit. The issue remains unresolved due to the user experience challenges associated with managing an encryption key for stored data.

Mitigation

We recommend adding a storage encryption provider responsible for supplying an encryption key for encrypting data at rest and for allowing other components to query the availability of this key. Encryption, decryption, and authentication of stored data should be handled by the core component of Midnight.js, with the storage encryption provider responsible only for supplying an encryption key and the private state provider responsible only for storing and retrieving encrypted data. This approach reduces the amount of security-critical code that application developers must implement if they choose to replace the default providers.

On mobile platforms, the storage encryption provider could delegate key generation and storage to the device's keychain. On server platforms, a key management service (KMS) could be used. Alternatively, storage of the encryption key could be delegated to the wallet, which already includes functionality for securely storing and backing up cryptographic keys, as well as managing access through mechanisms such as automatic time locks.

Status

The private state provider has been updated to encrypt data at rest. The encryption key is derived from a password using PBKDF2 with a random 32-byte salt and 600,000 rounds of SHA-256 hashing. The encryption algorithm is AES-256-GCM with a random 12-byte initialization vector per encrypted value. These algorithmic and parameter choices are suitable for this purpose. A version number is stored alongside the ciphertext, allowing the encryption parameters to be updated in the future without invalidating stored data.

Stored values that were previously stored without encryption are migrated to the new encrypted format opportunistically. An unencrypted value is detected by examining the first byte of the Base64-decoded data. In the new encrypted format, this byte contains the version number. Any byte other than the known version numbers 0x01 and 0x02 indicates that the value is unencrypted. This check could return the incorrect result if the first byte of an unencrypted value is 0x01 or 0x02. In that case, the unencrypted value would be treated as encrypted, but attempting to decrypt it would fail. We recommend removing opportunistic re-encryption of individual values and instead using a field in the sublevel's metadata structure to indicate whether the entire sublevel has been migrated to the encrypted format.

The provider allows the password from which the encryption key is derived to be updated. When this occurs, a new encryption key is derived from the new password, and all values in a given sublevel are read from the database, decrypted with the old key, re-encrypted with the new key, and written back to the database. The read and write operations are each atomic, but the overall read-and-write sequence is not. To prevent concurrent password rotation tasks from being interleaved, a lock is held while performing the read-and-write sequence. However, other operations that write to the database do not hold this lock. Some operations check whether the lock is free before performing their writes, but this is not sufficient to prevent their writes from being overwritten by a password rotation task. The following sequence of events would result in data being overwritten:

- Task A checks the lock and finds it free.
- Task A is suspended, and task B is started.
- Task B acquires the lock and begins password rotation.
- Task B atomically reads the values from the sublevel.
- Task B is suspended, and task A is resumed.
- Task A (which does not hold the lock) updates a value in the sublevel.
- Task A is suspended or finishes, and task B is resumed.
- Task B atomically writes the re-encrypted values to the sublevel, overwriting the update made by task A.

There are also two operations (`clear` and `clearSigningKeys`) that do not check or hold the lock. This could result in values that are cleared by these operations being written back to the database by a concurrent password rotation task, in a sequence of events similar to the one described above.

We recommend updating the locking code to require that all tasks writing to the database hold the lock. Due to the possibility of the encryption key changing between the time a task acquires the key and the time it uses the key to decrypt a value retrieved from the database, it may also be necessary for readers to hold the lock. If this adversely affects throughput, a multi-reader lock could be implemented.

We note that the provider as a whole does not appear to be designed for concurrent access: the `withSubLevel` method, which is used for most database operations, fails if called by concurrent tasks in the same process, as the underlying `Level` database does not allow a process to open a database that has already been opened and has not yet been closed. This indicates that concurrent calls to the provider are currently rare and that, accordingly, the concurrency risks outlined above may be unlikely to arise in

practice. However, the use of a lock suggests that the developers anticipate that concurrent calls to the provider may occur in the future.

Verification

Partially Resolved.

Issue B: Unproven Transaction Is Passed to Wallet Provider

Location

[packages/contracts/src/submit-tx.ts#L76](#)

[docs/image/call-tx-build-sequence-diagram.png](#)

Synopsis

The high-level documentation included in the repository indicates that transactions are proven before being passed to the wallet, whereas in practice the reverse is true. This discrepancy may lead to private data used in generating zero-knowledge proofs being unintentionally exposed to the wallet.

Impact

Low.

The wallet is expected to be trusted by the user for managing and storing blockchain accounts and cryptographic identities. Exposing additional private information to the wallet would therefore present a risk only if the wallet were compromised by an adversary. Preventing the wallet from accessing private data primarily serves as a defense-in-depth measure in the event that the wallet is compromised while other system components remain secure.

Feasibility

High.

Unproven transactions are passed to the wallet provider during the normal system operation. Although the implementation of the wallet provider lies outside the scope of the `midnight-js` repository, based on the version used in the integration tests, it is likely that the wallet provider forwards the transaction to the wallet without redacting any information.

Severity

Medium.

Technical Details

At a high level, Midnight.js prepares a transaction for submission by retrieving any required public and private data, executing the circuits used by the transaction, passing the result to a proof server that generates a zero-knowledge proof of the result, and then sending the proven transaction to the wallet for balancing. Finally, the proven and balanced transaction is submitted to the blockchain, and the new private state is stored if necessary. This process is documented in [a diagram included in the repository](#).

However, the implemented process differs slightly: the unproven transaction is passed to the wallet (via the wallet provider) for balancing before being sent to the proof server. As a result, any private state required for generating the zero-knowledge proof is passed to the wallet provider, and potentially to the wallet itself.

Mitigation

We recommend proving the transaction and redacting any unnecessary private data from the result before passing the proven transaction to the wallet provider. The redaction should be performed by the core component of Midnight.js, rather than the provider, to ensure that alternative provider implementations supplied by applications do not omit this step.

Alternatively, if the wallet requires access to private data from the unproven transaction in order to perform balancing, the documentation should be updated to accurately reflect that private data may be exposed to the wallet.

Status

The Midnight team has [updated](#) the `submitTxCore` function in the latest version of the `midnight-js` library to prove the transaction before passing it to the wallet.

Verification

Resolved.

Suggestions

Suggestion 1: Explicitly Disable Debugging Code by Default

Location

[packages/contracts/src/submit-tx.ts#L81](#)

Synopsis

When the `MN_DEBUG` environment variable is set, the `submitTx` function can throw an exception that may not be anticipated by its users and can also write potentially sensitive data to the user's filesystem.

Mitigation

We recommend clarifying the activation of `MN_DEBUG` to users. One approach would be to enable the debug section of the code only when the environment variable is set to a specific value, while by default assigning the variable to a different, disabled value. This would require users to explicitly change the variable to enable the potentially insecure debug behavior.

Status

The Midnight team has [updated](#) `midnight-js` to explicitly set a `__DEBUG__` variable according to the build environment as a dedicated build step, defaulting to true when built outside of CI.

Verification

Resolved.

Suggestion 2: Document the Errors Thrown by the Midnight.js API and Providers

Synopsis

The API documentation is generally of high quality, but the errors that may be thrown are not consistently documented.

Mitigation

We recommend thoroughly documenting all errors that can be thrown by the Midnight.js API so that applications using the framework can be written defensively to handle each type of error appropriately. Similarly, errors that can be thrown by the provider interfaces should be documented so that custom implementations of these providers can be developed to align with the behavior expected by other components of the framework.

Status

The Midnight team has [updated](#) pivotal functions of the `midnight-js` library to explicitly describe failure modes for wallet implementers. A representative example is the current implementation of the `submitTx` function.

Verification

Resolved.

Suggestion 3: Document the Behavior of Transaction Submission Methods When the Transaction Fails

Location

[packages/contracts/src/submit-call-tx.ts#L79](#)

[packages/contracts/src/submit-deploy-tx.ts#L67](#)

[packages/contracts/src/submit-insert-vk-tx.ts#L64](#)

[packages/contracts/src/submit-remove-vk-tx.ts#L58](#)

[packages/contracts/src/submit-replace-authority-tx.ts#L56](#)

Synopsis

A Midnight transaction is executed in two phases: the guaranteed phase and the fallible phase. If a transaction fails during the guaranteed phase, it is not included in the blockchain. If it fails during the fallible phase, the transaction is recorded on the blockchain as a partial success. The Midnight.js API provides methods for submitting various types of transactions, all of which throw errors if a transaction fails in either the guaranteed or fallible phase. For transactions that update private state or signing keys maintained by the private state provider, the updated state is not stored if a failure occurs during either phase.

Mitigation

For each transaction submission method, we recommend documenting the outcome of a failure at each phase, with particular attention to the errors thrown and the steps omitted (such as storing updated private state) in each case.

Status

The Midnight team has updated the `midnight-js` library documentation to explicitly state failure modes, including in [submit-call-tx.ts](#), [submit-deploy-tx.ts](#), [submit-insert-vk-tx.ts](#), [submit-remove-vk-tx.ts](#), and [submit-replace-authority-tx.ts](#).

Verification

Resolved.

Suggestion 4: Document the Behavior of Transaction Submission Methods When the Transaction Does Not Appear

Location

[packages/contracts/src/submit-tx.ts#L102](#)

Synopsis

The `submitTx` method, which submits transactions to the blockchain, relies on the public data provider's `watchForTxData` method, which waits indefinitely for the submitted transaction to appear on the blockchain. If a transaction reaches this stage of submission but is not published on the blockchain, for example due to being invalid in a way not previously detected, the `submitTx` method will never return. In practice, applications may eventually terminate the invocation of this method if it does not return.

Mitigation

We recommend documenting that the transaction submission methods do not return until the submitted transaction appears on the blockchain, along with any known conditions under which a transaction might fail to appear, and the implications of aborting each submission method in such cases (for instance, updated private state not being stored).

The documentation for the public state provider should also clearly define the expected behavior of this provider to reduce the risk of custom implementations deviating in this edge case (for example, by implementing a timeout rather than waiting indefinitely).

Status

The Midnight team has [updated](#) the `submitTx` function to properly document its behavior when the transaction does not appear.

Verification

Resolved.

Suggestion 5: Zeroize Sensitive Data to Avoid Accidental Exposure

Location

[packages/level-private-state-provider/src/level-private-state-provider.ts](#)

Synopsis

Highly sensitive information, such as private state, requires careful and deliberate handling. For this type of information, it is important to avoid leaving residual copies of the data in memory.

Mitigation

We recommend implementing a zeroizing mechanism to erase private information from memory after use. This approach overwrites the memory locations that stored the private information, ensuring that it is erased from memory. Libraries that may be used for this purpose include [sanitize-data](#) and [wasm-zeroize](#).

Status

The Midnight team has implemented deletion of sensitive data in the new private state provider component but does not sanitize it before deletion. As a result, the data may, in theory, remain resident in JavaScript runtime memory. Examples of deletion without sanitization include

[level-private-state-provider.ts#L232](#), [level-private-state-provider.ts#L265](#), and [level-private-state-provider.ts#L297](#).

Verification

Unresolved.

Suggestion 6: Use Monads to Distinguish Between Absent and Undefined Values

Location

[packages/level-private-state-provider/src/level-private-state-provider.ts#L109](#)

Synopsis

The `subLevelMaybeGet` method allows wallet implementers to retrieve private values from a key-value store. When library users call this function, two orthogonal pieces of information are retrieved and returned in a single value: first, whether a matching key exists in the store, and second, the value itself. This can create ambiguity for library users who may misinterpret a `null` return value as the absence of a key or an undefined return value as a key with an explicitly undefined value.

Mitigation

We recommend using an `Option<T>` type to represent this behavior more accurately and to provide clearer semantics for users regarding the function's return value. Adopting this approach would also remove the need for the function's implementation to remap undefined values to `null`, as users would be able to distinguish `Option.some(undefined)` from `Option.none` type.

Status

The Midnight team has [updated](#) the provider `get` method in the latest version of `midnight-js` to use `null` as a signifier that a key maps to undefined, does not exist, or has encountered other error conditions. Because this may create confusion for users of the API, we recommend adding documentation addressing the failure edge cases.

Verification

Unresolved.

Suggestion 7: Minimize Exposure of Private Data to Application Code

Location

[packages/contracts/src/tx-model.ts#L133](#)

[packages/contracts/src/submit-call-tx.ts#L88](#)

[packages/contracts/src/submit-deploy-tx.ts#L78](#)

Synopsis

Some methods in the Midnight.js API return objects that contain direct or indirect references to private data used in computing zero-knowledge proofs. This could result in application code unintentionally exposing such data, for instance, by logging or serializing returned objects. For example, the `FinalizedCallTxData` type, which represents the result of a call transaction, includes a reference to the `UnprovenTransaction` type, which may contain private data.

Mitigation

We recommend returning only the information strictly required by the application. When access to private data is necessary, applications should use the provider architecture to retrieve it explicitly. To support the consistent encapsulation of private data across the framework, TypeScript decorators could be applied to mark types that contain direct or indirect references to private data.

Status

The Midnight team has [updated](#) the `submit-call-tx` file in the latest version of `midnight-js` to interface with private data only through the providers rather than also exposing it in the return value.

However, the `submit-deploy-tx` still [returns](#) private info in the main path of the function call.

Verification

Partially Resolved.

Suggestion 8: Document the Framework's Threat Model

Synopsis

Application developers using the Midnight.js library would benefit from documentation that includes a clearly defined threat model established by the Midnight team. This would help library users more effectively assess and reason about the threat model of their own applications.

Mitigation

We recommend including documentation that defines the working threat model for the library. This document should describe how private data is handled within the library, outline the assumptions made about users' devices, and specify expectations regarding other agents with which the library interacts, such as Midnight nodes, peers, and external services. Additionally, it should clarify the expected behavior of provider implementers necessary for the library's threat model to remain valid. An example of a well-defined wallet SDK threat model can be found in the [Zcash Wallet App Documentation](#).

Status

At the time of verification, our team was unable to identify a threat model document for the latest version of `midnight-js`.

Verification

Unresolved.

About Least Authority

We believe that people have a fundamental right to privacy and that the use of secure solutions enables people to more freely use the Internet and other connected technologies. We provide security consulting services to help others make their solutions more resistant to unauthorized access to data and unintended manipulation of the system. We support teams from the design phase through the production launch and after.

The Least Authority team has skills for reviewing code in multiple Languages, such as C, C++, Python, Haskell, Rust, Node.js, Solidity, Go, JavaScript, ZoKrates, and circom, for common security vulnerabilities and specific attack vectors. The team has reviewed implementations of cryptographic protocols and distributed system architecture in cryptocurrency, blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, the team can utilize various tools to scan code and networks and build custom tools as necessary.

Least Authority was formed in 2011 to create and further empower freedom-compatible technologies. We moved the company to Berlin in 2016 and continue to expand our efforts. We are an international team that believes we can have a significant impact on the world by being transparent and open about the work we do.

For more information about our security consulting, please visit <https://leastauthority.com/security-consulting/>.

Our Methodology

We like to work with a transparent process and make our reviews a collaborative effort. The goals of our security audits are to improve the quality of systems we review and aim for sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Manual Code Review

In manually reviewing all of the code, we look for any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and whitebox penetration testing. We look at the project's website to get a high level understanding of what functionality the software under review provides. We then meet with the developers to gain an appreciation of their vision of the software. We install and use the relevant software, exploring the user interactions and roles. As we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation. We hypothesize what vulnerabilities may be present and possibly resulting in Issue entries, then for each, we follow the following Issue Investigation and Remediation process.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and seeing them through successful remediation. Whenever a potential issue is discovered, we immediately create an Issue entry for it in this document, even before having verified the feasibility and impact of the issue. This process is conservative because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyze the feasibility of an attack in a live system.

Suggested Solutions

We search for immediate and comprehensive mitigations that live deployments can take, and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinized by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our Initial Audit Report, and before we perform a verification review.

Before our report, including any details about our findings and the solutions are shared, we like to work with your team to find reasonable outcomes that can be addressed as soon as possible without an overly negative impact on pre-existing plans. Although the handling of issues must be done on a case-by-case basis, we always like to agree on a timeline for a resolution that balances the impact on the users and the needs of your project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess that the issues and suggestions are sufficiently addressed. When this analysis is completed, we update the report and provide a Final Audit Report that can be published in whole. If there are critical unaddressed issues, we suggest the report not be published and the users and other stakeholders be alerted of the impact. We encourage that all findings be dealt with and the Final Audit Report be shared publicly for the transparency of efforts and the advancement of security learnings within the industry.